Distributed Systems' Project
Ant Colony Optimization Problem

Daniele Ferroli (mat. 137357)
Luca Gemolotto (mat. 143865)
DMIF, University of Udine, Italy

16 agosto 2023

Indice

1 Introduzione **1**
- 1.1 Descrizione del problema . . . 1
  - 1.1.1 Funzionalità del sistema . . . 1
- 1.2 Struttura dell'applicazione . . . 1
  - 1.2.1 Struttura distribuita . . . 2
  - 1.2.2 Algoritmi e CAP theorem . . . 2
- 1.3 Trasparenze del sistema . . . 3
- 1.4 Piano di testing . . . 4
- 1.5 Piano di sviluppo . . . 4

2 Analisi dei requisiti **5**
- 2.1 Requisiti funzionali . . . 5
  - 2.1.1 Descrizione delle funzioni . . . 5
- 2.2 Requisiti non funzionali . . . 6

3 Project **7**
- 3.1 Logical architecture . . . 7
- 3.2 Protocols and algorithms . . . 7
- 3.3 Physical architecture and deployment . . . 7
- 3.4 Development plan . . . 7

4 Implementation **8**

5 Validation **9**

6 Conclusions **10**

A Appendix **11**

Sommario

Capitolo 1

Introduzione

1.1 Descrizione del problema

Il problema analizzato consiste nel fornire in tempo reale a un agente un'approssimazione del cammino migliore per i suoi clienti. La topologia dei clienti viene fornita dall'utente in modo tale da poterla aggiornare in base alle sue necessità. A tal fine viene proposta una soluzione basata sull'algoritmo di "Ant Colony Optimization" (**ACO**).

1.1.1 Funzionalità del sistema

Illustriamo a seguire la nostra proposta:

- l'utente può interfacciarsi al sistema per inserire i clienti e i costi per spostarsi tra quest'ultimi;
- il sistema crea un grafo e computa un'approssimazione del percorso migliore;
- tramite l'interfaccia grafica viene visualizzato il grafo e il percorso;
- l'utente può fornire nuovi dati sui clienti e/o i costi;
- il sistema si aggiorna mostrando il nuovo percorso all'agente.

Il sistema si presuppone di essere in continua esecuzione e che l'approssimazione del percorso migliore sia adeguata.

Infine, la soluzione proposta è applicabile anche ad altri contesti, con pochi adattamenti necessari, se non nessuno. Uno di questi contesti potrebbe essere un sistema di veicoli autonomi, in cui quest'ultimi si muovono all'interno di un'area, comunicandosi informazioni riguardanti lo stato dei percorsi e dove trovare i percorsi migliori.

1.2 Struttura dell'applicazione

L'applicazione è composta dai seguenti elementi:

- il sistema ACO distribuito è il modulo logico per computare l'approssimazione del TSP;
- un'interfaccia grafica per poter visualizzare il grafo e il cammino calcolato;
- un modulo intermedio tra sistema ACO, l'interfaccia grafica e le interazioni dell'utente (API);
- un agente (o multipli) che utilizza l'interfaccia grafica;
- un utente (o un sistema) che interagisce con il modulo intermedio, per aggiungere o rimuovere clienti e modificare informazioni sui percorsi.

Da questo punto in poi, con *agente* si intenderà l'ente che usa l'interfaccia grafica, mentre con *utente* si intende solamente chi utilizza le API del sistema.

1.2.1 Struttura distribuita

Il sistema ACO è un sistema distribuito peer-to-peer, i cui nodi non necessariamente hanno una conoscenza completa della topologia. All'interno del sistema possono essere presenti più nodi su una stessa macchina (fisica o virtuale).

I nodi, idempotenti, possono essere di due tipi:

- nodo utente: aggiorna il grafo dei clienti e riceve informazioni sul grafo;
- nodo ACO: gestisce i nodi del grafo dei clienti e partecipa alla computazione dell'algoritmo.

La comunicazione tra nodi è **asincrona** e **accoppiata** rispetto sia al **tempo** che allo **spazio**. Questo perché un nodo invia un messaggio a un suo vicino (specifico), il quale deve esistere nello stesso momento, senza però dover aspettare una risposta.

La topologia del sistema è rappresentata da un grafo pesato non orientato composto da clienti in cui gli archi sono le effettive distanze più eventuali perturbazioni, come ad esempio lo stato del traffico. I nodi del sistema ACO gestiscono un sottoinsieme dei clienti presenti.

1.2.2 Algoritmi e CAP theorem

Considerando il CAP theorem, si è scelto di assicurare disponibilità (**availability**) e tolleranza rispetto alle partizioni (**partition tolerance**). Questo perché è necessario che il sistema sia sempre responsivo e possa funzionare anche se un nodo è soggetto a malfunzionamenti. Nel caso di un'azienda di dimensioni ridotte sarebbe preferibile implementare un sistema consistente piuttosto che disponibile, poiché il grafo dei clienti non riceverebbe frequenti aggiornamenti. Nel nostro caso, invece, supponiamo di lavorare per un'azienda sufficientemente grande, con una discreta dinamicità dei clienti. Risulta quindi più importante poter aggiornare il grafo velocemente rispetto al fatto che l'intero sistema abbia le informazioni più recenti.

Al fine di visualizzare il grafo ACO globale, tramite interfaccia grafica, è necessario che tutti i nodi pubblicizzino i propri dati. È stato quindi pensato di utilizzare una variante dell'ECHO algorithm. Un nodo utente viene notificato da un nodo ACO di un round finito e propaga l'informazione tramite echo a tutti

gli altri nodi utente. L'algoritmo viene usato anche durante l'inizializzazione del sistema per informare tutti i nodi utente dei nodi del grafo. Infine, esso viene anche utilizzato per l'aggiunta e rimozione di nodi del grafo.

Per aggiornare il grafo dei clienti, è stato scelto di utilizzare un sistema a *ticket*. L'utente, per poter effettuare un aggiornamento, deve chiedere un ticket. Verrà quindi mandato un messaggio al nodo ACO a cui è collegato, che lo inoltrerà al *ticket master*. Il ticket master risponderà quindi inviando il ticket, che l'utente potrà usare per fare richiesta di aggiornare il grafo. Questo sistema serve per assicurare che gli aggiornamenti vengano fatti in ordine, ad esempio se un utente decide di aggiungere un nodo, non deve essere possibile che un altro utente possa rimuoverlo prima che questo aggiornamento sia noto a tutti i nodi ACO.

Il ticket master è un nodo ACO che viene eletto tramite un algoritmo echo con extinction. Ogni nodo ACO inizia un echo, allegando il proprio ID e scarta tutti gli "echi" con ID minore al proprio. Se invece riceve un echo con ID maggiore, terminerà il proprio e parteciperà a quello appena ricevuto. Alla fine, tutti gli "echi" tranne quello con ID maggiore verrano terminati senza successo e l'iniziatore sarà il leader.

ECHO algorithm per mandare dati formiche fine round a tutti i nodi. No lista di nodi conosciuti, formica non può venire mandata in nodi già visitati, non può venire mandata al nodo iniziale se ci sono archi a nodi non visitati (anche fittizi). {deadlock, ANT, VISITED_AGENTS} ¡- messaggio in caso formica non possa muoversi su archi "normali".

1.3 Trasparenze del sistema

È stato scelto di implementare le seguenti trasparenze:

- *access transparency*: l'utente può aggiungere o rimuovere nodi del grafo senza sapere se questi si trovano in locale o in remoto;
- *location transparency*: l'utente non ha conoscenza della posizione fisica e/o della posizione nella rete degli altri nodi;
- *concurrency transparency*: i nodi operano sullo stesso grafo in contemporanea senza interferire tra loro;
- *scaling transparency*: nuovi nodi possono essere aggiunti al sistema senza che questo subisca variazioni o interruzioni;
- *failure transparency*: terminazioni inaspettate di nodi del sistema non portano a interruzioni del servizio per l'utente.

Le rimanenti trasparenze non sono state implementate perché non inerenti alla richiesta.

loopACO: if userQueue =!= empty; msg = pop.userQueue(); dealWithUser(msg); for n in neighbourList n ! msg; if

1.4 Piano di testing

Il sistema verrà testato su diversi grafi casuali, statici e dinamici. Verrà inoltre testata la resilienza del sistema a cambiamenti della topologia e interruzioni inaspettate dei nodi.

Altro oggetto di testing sarà l'osservazione di quanti round (per nodo), in media, sono necessari per raggiungere la convergenza al percorso migliore.

1.5 Piano di sviluppo

Lo sviluppo seguirà le seguenti fasi:

- progettazione del sistema e dei casi d'uso;
- specificazione dei modelli di comunicazione del sistema;
- progettazione interfaccia grafica;
- progettazione dell'implementazione del sistema ACO e dell'interfaccia;
- scrittura casi di test;
- implementazione in Erlang del sistema ACO;
- testing sistema ACO;
- implementazione dell'interfaccia grafica;
- testing e validazione finale.

Capitolo 2

Analisi dei requisiti

In questo capitolo verranno descritti i requisiti funzionali e non funzionali del sistema.

2.1 Requisiti funzionali

- aggiunta e rimozione dei clienti: l'utente deve poter aggiungere e rimuovere clienti nel sistema, rappresentati come nodi del grafo;
- aggiornamento costo spostamento tra clienti: l'utente deve poter aggiornare il peso degli archi, che rappresenta il costo di spostarsi da un cliente all'altro. Questo costo può tenere conto sia della condizione dei percorsi, che il costo in termini di tempo e denaro;
- visualizzazione algoritmo ACO in tempo reale: l'utente deve poter visualizzare le approssimazioni del camminino migliore, mentre l'algoritmo è in esecuzione;
- uso di grafi completi: è stato richiesto che l'implementazione usi un grafo completo, anche se quello fornito dal cliente non lo sia. In questo caso il grafo viene completato con archi *fittizi* con pesi sufficientemente elevati. Non si vuole che i pesi siano infiniti ma neppure che sia così bassi che possano venire presi al posto di un arco con peso *reale*.

2.1.1 Descrizione delle funzioni

- Aggiunta dei clienti
  - Input: tripla formata da identificatore del cliente, identificatore della sede relativa a questo, lista dei costi per spostarsi dal cliente ad altri clienti.
  - Output: conferma da parte del sistema dell'aggiunta, o messaggio di errore nel caso in cui l'operazione non sia andata a buon termine.
  - Effetto: aggiunta del nodo cliente all'interno della memoria locale del nodo ACO che fa riferimento alla sede `Sede`, con relativi archi.
- Rimozione dei clienti

  - Input: identificatore del cliente da rimuovere.
  - Output: conferma da parte del sistema della rimozione, o messaggio di errore nel caso in cui l'operazione non sia andata a buon termine.
  - Effetto: rimozione del nodo cliente dalla memoria locale del nodo ACO che lo conteneva e rimozione archi che comprendono il cliente.

- Aggiornamento costo spostamento tra clienti
  - Input: ⟨ `[{Cliente1, Cliente2, costo }]` ⟩
  - Output: conferma della da parte del sistema dell'aggiornamento, o messaggio di errore nel caso in cui l'operazione non sia andata a buon termine.
  - Effetto: aggiornamento dei costi degli archi per ogni coppia `(Cliente1, Cliente2)` nei relativi nodi ACO.

- Visualizzazione grafo
  - Input: nessuno
  - Output: grafo aggiornato in tempo reale
  - Effetto: visualizzazione da parte dell'utente del grafo aggiornato in tempo reale.

2.2 Requisiti non funzionali

È stato deciso di utilizzare il linguaggio Erlang per l'implementazione del sistema.

Le trasparenze del sistema saranno come descritte nella sezione 1.3.

Non sono stati richiesti limiti per quanto concerne prestazioni o tempistiche del tempo, ma è stato chiesto di usare una versione asincrona dell'algoritmo ACO, se possibile. Non sono stato effettuate richiesta rispetto alla sicurezza del sistema. A tale scopo, si è individuata una variante asincrona, presentata in [1], come punto di partenza.

I clienti sono identificati da ID unici e non è possibile aggiungere multipli clienti con lo stesso ID.

Capitolo 3

Project

This chapter is devoted to the description of the general architectures, and specific algorithms.

3.1 Logical architecture

Describe the components of your systems: modules/objects/components/services. For each component, describe the functionalities it implements, and by who is used.

3.2 Protocols and algorithms

Communication between components. UML sequence diagrams go here.

Also, put here a detailed description of distributed algorithms used to solve specific problems of the project.

3.3 Physical architecture and deployment

Which nodes and platforms involved, and where each component is deployed.

3.4 Development plan

Since it is difficult to predict just how hard implementing a new system will be, you should formulate as a set of "tiers," where the basic tier is something you're sure you can complete, and the additional tiers add more features, at both the application and the system level.

Capitolo 4

Implementation

Details about the implementation: every choice about platforms, languages, software/hardware, middlewares, which has not been decided in the requirements.

Important choices about implementation should be described here; e.g., peculiar data structures.

Capitolo 5

Validation

Check if requirements from Chapter 2 have been fulfilled. Quantitative tests (simulations) and screenshots of the interfaces are put here.

Capitolo 6

Conclusions

What has been done with respect to what has been promised in Chapters 1 and 2, and what is left out.

Appendice A

Appendix

In the Appendix you can put code snippets, snapshots, installation instructions, etc.

Evaluation

Your system will be judged mainly on how it operates as a distributed system. The primary evaluation will be according to whether your system has the following attributes:

- It should be an interesting distributed system, making use of some of the algorithms we have covered in class for distributed synchronization, replication, fault tolerance and recovery, security, etc.
- The software should be well designed and well implemented, in terms of the overall architecture and the detailed realization.
- You should devise and apply systematic testing procedures, at both the unit and systems levels.
- The system should operate reliably and with good performance, even in the face of failures.

Important, but secondary considerations include:

- Time taken to do the project (the sooner the better, but do not miss details in order to end sooner)
- How nice is the application's appearance: does it have a nice interface or a compelling visual display?

Bibliografia

[1] Sorin Ilie and Costin Bădică. Multi-agent approach to distributed ant colony optimization. *Science of Computer Programming*, 78(6):762–774, 2013.